

AI HEALTHCARE SYSTEM

Complete Project Documentation

Generated: May 29, 2026

TABLE OF CONTENTS

1. Project Overview
2. Architecture & Stack
3. Features by Role
4. Data Flow & Collection
5. Light Mode Implementation
6. Database Schema
7. Bug Fixes Applied
8. API Endpoints
9. Security & Authentication
10. Deployment

1. PROJECT OVERVIEW

AI Healthcare System is a comprehensive web-based medical management platform that combines:

- **Patient Management:** Health records, appointments, prescriptions tracking
- **Doctor Management:** Patient assignments, medical record reviews, appointment scheduling
- **Admin Control:** System-wide management, user administration, analytics
- **AI Integration:** Symptom-based diagnosis prediction with confidence scoring
- **Real-time Updates:** Notifications, status tracking, live data synchronization

2. ARCHITECTURE & STACK

Frontend:

Next.js 14+ | React 19 | TypeScript | Tailwind CSS | Framer Motion

Backend:

FastAPI | Python | SQLAlchemy ORM | Pydantic

Database & Services:

PostgreSQL 16 | Redis 7 | Docker Compose | JWT Authentication

3. FEATURES BY ROLE

PATIENT Features:

- | | |
|------------------------------------|---------------------------|
| ✓ View personal health records | ✓ Book appointments |
| ✓ View medications & prescriptions | ✓ Track diagnosis history |
| ✓ Receive notifications | ✓ Manage health profile |

DOCTOR Features:

- | | |
|--------------------------|--------------------------|
| ✓ View assigned patients | ✓ Update medical records |
| ✓ Manage appointments | ✓ Prescribe medications |
| ✓ Review diagnoses | ✓ Track patient progress |

ADMIN Features:

✓ Manage all users

✓ Track active cases

✓ Global search

✓ View all medical records

✓ System analytics

✓ Appointment management

4. DATA FLOW & COLLECTION

Real-time Data Collection Process:

User Action → Frontend API Call → Backend FastAPI Endpoint → Database (PostgreSQL) → Redis Cache Update → Response to Frontend → UI Update

Data Collection Methods:

1. **Direct User Input:** Appointments, Symptoms, Profile Updates
2. **System Auto-Tracking:** Status Updates, Doctor Assignments, Notifications
3. **Real-time Monitoring:** Attendance, Live Updates, Schedule Changes

Key Data Points Collected:

- User registrations and authentications
- Medical records and diagnoses
- Appointment bookings and completions
- Prescription dispensing and adherence
- Doctor assignments and reassignments
- System notifications and audit logs

5. LIGHT MODE IMPLEMENTATION

Color System:

Primary Header Color: Royal Blue (#4169E1)

Text Color: Black (#000000)

Background: White/Light Gray

Implementation Approach:

- All dashboard headers use consistent royal blue color
- CSS variables for dynamic theme switching
- Tailwind CSS utility classes for responsive design
- Dark text on light backgrounds for accessibility

Pages with Light Mode:

- ✓ Dashboard (Overview)
- ✓ Medications
- ✓ My Patients (Doctor view)
- ✓ Patients (Admin view)
- ✓ Medical Records
- ✓ Active Cases
- ✓ Appointments
- ✓ Admin Dashboard

6. DATABASE SCHEMA

Core Tables:

users - id, email, hashed_password, role, is_active, created_at

doctor_profile - user_id, specialization, availability_status

patient_profile - user_id, blood_group, chronic_conditions, emergency_contact

Medical Data:

medical_records - id, patient_id, ai_prediction, confidence_score, status, doctor_id

appointments - id, patient_id, date, time, status, specialist, doctor_id

prescriptions - id, patient_id, medication_name, dosage, frequency, status

System Tables:

notifications - id, user_id, type, title, message, created_at

audit_logs - id, user_id, action, resource, timestamp

doctor_patient - doctor_id, patient_id, assignment_date, status

7. BUG FIXES APPLIED

Bug 1: Past Appointments Showing 'Upcoming'

Description: Appointments with past dates were not updating to 'Completed' status

Root Cause: Auto-update logic only processed paginated results (50 item limit)

Solution: Database-level query to update ALL past appointments before pagination

Bug 2: Doctor Seeing All Patients

Description: All doctors could see all patients' medical records

Root Cause: doctor_id was NULL when records were created

Solution: Auto-assign doctor_id at record creation based on specialty matching

Bug 3: Admin Dashboard Active Cases = 0

Description: Active cases counter displayed 0 despite having 124 active records

Root Cause: Filter checked for 'pending' status but database uses 'in_review'/'reviewed'

Solution: Corrected filter logic and increased query limit from 100 to 1000

Bug 4: Light Mode Headers Not Visible

Description: White text on white background in light mode

Solution: Changed all headers to royal blue (#4169E1)

Bug 5: Unnecessary Search Functionality

Description: Search inputs scattered across dashboards

Solution: Removed search from all pages (medications, my-patients, patients, records, search)

8. KEY API ENDPOINTS

Authentication:

POST /api/v1/auth/login - User login
POST /api/v1/auth/register - User registration
GET /api/v1/auth/me - Current user info

Medical Records:

GET /api/v1/records - Get user records
POST /api/v1/records - Create new record
GET /api/v1/records/{id} - Get specific record
PUT /api/v1/records/{id} - Update record

Appointments:

GET /api/v1/appointments - List appointments
POST /api/v1/appointments - Book appointment
PUT /api/v1/appointments/{id} - Update appointment
DELETE /api/v1/appointments/{id} - Cancel appointment

Prescriptions:

GET /api/v1/prescriptions - Get prescriptions
POST /api/v1/prescriptions - Add prescription
PUT /api/v1/prescriptions/{id} - Update prescription

9. SECURITY & AUTHENTICATION

Authentication Method: JWT (JSON Web Tokens)

Storage: HttpOnly Cookies (prevents XSS attacks)

Password: Hashed with secure algorithm

Authorization:

- Role-based access control (RBAC)
- Patient: Can only view own data
- Doctor: Can only view assigned patients
- Admin: Full system access

Data Protection:

- Database encryption for sensitive data
- Rate limiting on authentication endpoints
- Audit logs for all actions
- HTTPS/SSL for all communications

10. DEPLOYMENT & INFRASTRUCTURE

Containerization: Docker Compose

Services:

- Frontend Container (Next.js on port 3006)
- Backend Container (FastAPI on port 8006)
- PostgreSQL Database (port 5432)
- Redis Cache (port 6379)

Build Process:

1. Frontend: npm install → npm run build
2. Backend: pip install → uvicorn app.main:app
3. Database: PostgreSQL initialization with schema
4. All services via docker-compose up -d

Deployment Status: ✓ All containers running

Git Status: ✓ All 10 commits pushed to main

Build Status: ✓ No TypeScript compilation errors

Last Updated: May 29, 2026 at 10:37 PM

Commits Pushed:

1. 740a053 - Disable search functionality on global search dashboard
2. d319c7b - Remove search functionality from records dashboard
3. a385997 - Remove search functionality from my-patients and patients dashboards
4. 56931e9 - Fix: Remove remaining reference to search variable
5. b546e72 - Fix admin dashboard active cases counter showing 0
6. 58ba64d - Remove search functionality from medications dashboard
7. e6c03dc - Fix doctor seeing all patients: Auto-assign doctor to medical records
8. 14374e6 - Fix appointment status logic: auto-update ALL past appointments
9. Plus 2 more historical commits